

DMF Benchmarks - user manual

Configuring experiments and operating `dmf-benchctl`

DMF Benchmarks

August 11, 2026

1. Purpose

This manual explains how to configure and operate DMF Benchmarks: how to prepare a configuration, validate it, run a benchmark, read the results, and publish an official run bundle.

The document is not tied to a single smoke dataset. Examples use concrete paths so they can be copied, but the flow is the same for LoCoMo, LongMemEval, DMF, and Mem0.

The system has two command-line surfaces with separate responsibilities. Users operate `dmf-benchctl` on the host. The controller manages the current orchestrator and explicitly starts `dmf-bench` inside the benchmark image. `dmf-bench` remains the scientific runtime; it does not start infrastructure itself.

All benchmarks are native-only and run inside Docker. The old protocol concept no longer exists.

2. Components

A standard execution uses these components:

- `dmf-benchctl`: the host-side control plane used by operators and automation.
- `benchmark`: the container that runs `dmf-bench`.
- `qdrant`: the local vector database used by the frameworks.
- `artifact-api`: a read-only API for reading runs and artifacts.
- `/bench/config`: read-only mounted configurations.
- `/bench/cache`: writable cache for materialized datasets, models, and temporary data.
- `/bench/runs`: writable volume where results are published.

Provider keys must never be written into JSON files. Declare them in the host environment or repository `.env`, for example as `OPENAI_API_KEY`. `dmf-benchctl` copies each required value into a private ephemeral file and mounts it read-only for the benchmark process. The value is not included in Compose service environment metadata or command arguments, and the temporary host file is deleted when the job command finishes.

3. Prerequisites

You need Docker with the Compose plugin, Python 3.12 or newer, the repository files, and the `dmf-benchctl` command installed from this project.

From the repository root, install the lightweight controller with `pipx`:

```
python3 -m pip install --user pipx
python3 -m pipx ensurepath
pipx install .
```

Open a new shell if `ensurepath` requests it. The `dmf-benchctl` executable is then available directly: user commands do not need a `poetry run` prefix. This default installation does not install DMF, Mem0, OpenAI, embedding models, or the scientific runtime dependencies because those belong to the Docker image.

For editable controller development, use `pipx install --editable ..`. Repository maintainers who run Python unit tests should instead install the `runtime` extra through the documented Makefile/Poetry workflow.

Then check the two required host surfaces:

```
docker version
dmf-benchctl --help
```

If you use an image published to GHCR:

```
docker login ghcr.io
```

Official RC and tagged image references contain both `linux/amd64` and `linux/arm64`. Docker selects the native variant automatically. On an Apple Silicon Mac, verify that the pulled image reports `linux/arm64` before recording official timing measurements; an emulated `linux/amd64` image can produce misleading timings.

Create a local `.env` file in the repository root:

```
DMF_BENCH_IMAGE=ghcr.io/ORG/IMAGE:TAG
OPENAI_API_KEY=paste_the_key_here
HF_HUB_OFFLINE=1
```

`HF_HUB_OFFLINE=1` prevents implicit Hugging Face model downloads during an official run. If the cache does not already contain the required models, the run may fail; prepare the cache before the run.

No Qdrant API key is needed when Qdrant runs locally without authentication.

The `.env` file is read by `dmf-benchctl`. You can also export the same variables in the shell. Never commit the file.

The benchmark container defaults to four CPUs and four worker threads. For a different controlled limit, add values such as these to `.env`:

```
DMF_BENCH_CPUS=6
DMF_BENCH_THREADS=6
```

Use identical limits for every framework in a timing comparison.

4. Configuration

An experiment is described by one JSON file with `schema_version: 2`.

The configuration is the scientific contract of a run. It says what benchmark is executed, which framework is evaluated, which dataset is used, which model answers, which model judges, where state is stored, and which reports must be produced.

A config should be treated as immutable once a run is considered official. If you change the dataset, framework config, selected units, model names, model parameters, or evaluation requirements, you are defining a different experiment.

The top-level structure is:

- `schema_version`: must be 2. Older benchmark config formats are rejected.
- `experiment_id`: stable human-readable experiment name. It should describe the benchmark, framework, and scope.
- `benchmark`: task to execute, either `locomo` or `longmemeval`.
- `framework`: memory framework under test, either `dmf` or `mem0`.
- `runtime`: operational paths and process settings. This includes root directory, run directory, cache directory, metrics port, and log level.
- `framework_config`: framework-specific TOML or YAML file plus SHA-256. This pins the DMF/Mem0 runtime configuration used by the experiment.
- `qdrant`: how the benchmark reaches Qdrant. The URL is read from an environment variable, normally `QDRANT_URL`.

- **dataset**: dataset declaration. It can point to an already materialized JSON file or to a registry entry that `dmf-bench` materializes before the run.
- **selection**: exact benchmark units to execute and optional filters. This controls the experiment size and order.
- **models**: provider, requested model, generation parameters, timeout, rate limit, and retry policy for answerer and judge.
- **evaluation**: reports that must or may be produced. Missing required reports make the run fail.
- **artifact_store**: destination for committed artifacts, normally the same directory as `runtime.runs_dir`.

The most important rule is separation of responsibilities:

- JSON config files define reproducible experiment inputs.
- Environment variables provide secrets and deployment endpoints.
- Run artifacts record what actually happened.

Do not put provider API keys in the JSON config. For example, `models.answerer.provider` may be `openai`, but the actual key must come from `OPENAI_API_KEY`. Similarly, `qdrant.endpoint_env` normally contains the string `QDRANT_URL`; the actual URL is read from that environment variable at runtime.

Path rules

When running inside the official Docker image, paths normally live under `/bench`:

```
/bench/config    read-only configuration files
/bench/cache     writable cache for datasets, models, and temporary data
/bench/runs      writable output directory for committed run artifacts
```

Paths used by the benchmark must be absolute inside the runtime environment and must stay under `runtime.root`, normally `/bench`. This prevents a config from accidentally reading or writing unrelated host files.

In commands, `dmf-benchctl` accepts either a host path or an existing `/bench/...` path for the experiment config. Repository paths under `config/` and `smoke/` are translated automatically. A config elsewhere is mounted read-only at `/bench/operator/<filename>`. Paths written inside the JSON still describe the container runtime and therefore normally remain under `/bench`.

For example:

```
{
  "runtime": {
    "root": "/bench",
    "runs_dir": "/bench/runs",
    "cache_dir": "/bench/cache",
    "metrics_port": 9464,
    "log_level": "INFO"
  },
  "artifact_store": {
    "type": "local",
    "uri": "/bench/runs"
  }
}
```

Scientific versus operational fields

Some fields define the scientific identity of the run. Changing them changes the experiment:

- benchmark
- framework
- framework_config.sha256
- dataset

- selection
- models.answerer.requested_model
- models.answerer.parameters
- models.judge.requested_model
- models.judge.parameters
- evaluation

Other fields are operational. They can change where or how the same experiment is executed, but they should still be recorded for auditability:

- runtime.runs_dir
- runtime.cache_dir
- runtime.metrics_port
- runtime.log_level
- qdrant.request_timeout_seconds
- artifact_store.uri

dmf-bench records both kinds of information in the run manifest and provenance artifacts, but the scientific fingerprint focuses on the scientific inputs.

Minimal shape

A complete config has this shape:

```
{
  "schema_version": 2,
  "experiment_id": "locom-dmf-official-run",
  "benchmark": "locom",
  "framework": "dmf",
  "runtime": {
    "root": "/bench",
    "runs_dir": "/bench/runs",
    "cache_dir": "/bench/cache",
    "metrics_port": 9464,
    "log_level": "INFO"
  },
  "framework_config": {
    "path": "/bench/config/locom_dmf_qdrant_settings.toml",
    "sha256": "64_character_sha256",
    "format": "toml",
    "profile": "official-v1"
  },
  "qdrant": {
    "endpoint_env": "QDRANT_URL",
    "retention": "delete-on-success",
    "request_timeout_seconds": 10
  },
  "dataset": {
    "name": "locom",
    "registry_id": "locom-official-v1"
  },
  "selection": {
    "ordered_item_ids": ["conversation-0001"],
    "filters": {},
    "seed": 7
  },
  "models": {
    "answerer": {
```

```

    "provider": "openai",
    "requested_model": "gpt-4.1-mini",
    "parameters": {
        "temperature": 0,
        "max_tokens": 4096
    },
    "runtime": {
        "timeout_seconds": 120,
        "rpm": 200,
        "max_retries": 5
    }
},
"judge": {
    "provider": "openai",
    "requested_model": "gpt-5-mini",
    "parameters": {
        "temperature": 0,
        "max_tokens": 4096,
        "reasoning_effort": "low"
    },
    "runtime": {
        "timeout_seconds": 120,
        "rpm": 200,
        "max_retries": 5,
        "response_max_retries": 1
    }
}
},
"evaluation": {
    "required": ["primary_judge_score", "rigorous_report"],
    "optional": ["ablation_report"],
    "ablation_required": false
},
"artifact_store": {
    "type": "local",
    "uri": "/bench/runs"
}
}

```

Before running an experiment, validate the config through the controller:

```
dmf-benchctl validate --materialize-datasets --config path/to/experiment.json
```

The controller mounts or translates the host path and runs `dmf-bench validate` inside the same image used for execution. Validation checks the config schema, path boundaries, framework config hash, dataset declaration, and environment-dependent settings that can be checked without executing the benchmark lifecycle.

`qdrant.retention` is operational. Use `delete-on-success` for official batch runs: after all predictions for one atomic input record are complete, its owned Qdrant collections are deleted before that record is committed. If deletion fails, the record remains resumable and is not reported as committed. Use `keep` only when retained collections are intentionally needed for debugging.

`models.*.runtime.max_retries` controls retryable transport failures. `models.judge.runtime.response_max_retries` separately controls malformed or truncated judge responses. A value of 1 allows one corrective retry; all attempt tokens are counted, and exhausting the limit fails closed instead of inventing a score.

5. Dataset

`dmf-bench` supports two ways to declare a dataset.

Already materialized dataset

Use this form when the dataset JSON file is already available:

```
{
  "dataset": {
    "name": "locomo",
    "source": "local-approved-dataset",
    "revision": "v1",
    "path": "/bench/datasets/locomo/locomo10.json",
    "sha256": "64_character_sha256",
    "expected_schema": "locomo10-json-array-v1"
  }
}
```

In this mode, `dmf-bench validate` checks that the file exists and that the SHA-256 matches.

Registry dataset

Use this form when you want `dmf-bench` to download or reuse a registered source:

```
{
  "dataset": {
    "name": "locomo",
    "registry_id": "locomo-official-v1"
  }
}
```

During `run`, the dataset is materialized under:

`/bench/cache/datasets/<fingerprint>/`

The directory always contains a `manifest.json`. The manifest documents:

- dataset id, benchmark, source URL, and revision;
- path and SHA-256 of the observed source file;
- path and SHA-256 of the materialized file;
- file sizes;
- optional sampling policy.

Official registry entries are immutable: their URL contains an upstream commit and their bytes must match the registered SHA-256. Custom exploratory registries may still declare unpinned sources, but those are not suitable for prepared or official runs.

6. Sampling

Sampling is optional and is declared inside `dataset.sampling`. To reduce the work performed by both ingestion and answering, sample complete top-level dataset records rather than questions inside otherwise complete records.

LoCoMo example, 5% of complete conversations:

```
{
  "dataset": {
    "name": "locomo",
    "registry_id": "locomo-official-v1",
    "sampling": {
```

```

    "fraction": 0.05,
    "unit": "conversation",
    "seed": 7,
    "rounding": "ceil"
  }
}

```

Conversation sampling keeps every session and QA item belonging to each selected conversation. LoCoMo contains only 10 top-level conversations, so a 5% sample with `ceil` rounding selects one complete conversation. The effective conversation fraction is therefore 10%; selecting half a conversation would not preserve the dataset record. The materialization manifest records both conversation and question counts so the effective sample is explicit.

LoCoMo also supports `unit: "qa"` with optional `stratify_by: "category"` for question-focused analyses. That mode can retain many or all conversations and is therefore not appropriate when the purpose is to reduce ingestion in the same proportion as answering.

LongMemEval example, 5% of records:

```

{
  "dataset": {
    "name": "longmemeval",
    "registry_id": "longmemeval-s-official-v1",
    "sampling": {
      "fraction": 0.05,
      "unit": "record",
      "seed": 7,
      "rounding": "ceil"
    }
  }
}

```

Each selected LongMemEval record carries its complete memory and question, so the same materialized 5% sample bounds both ingestion and answering.

If sampling is omitted, `manifest.json` is still generated, but without a `sampling` block.

7. Selection

`selection.ordered_item_ids` decides which units are executed and in which order.

For LoCoMo, IDs are conversations, for example:

```

{
  "selection": {
    "ordered_item_ids": ["conversation-0001", "conversation-0002"],
    "filters": {"categories": [1, 2, 3, 4, 5]},
    "seed": 7
  }
}

```

For LongMemEval, IDs are `question_id` values, for example:

```

{
  "selection": {
    "ordered_item_ids": ["10d9b85a", "gpt4_45189cb4"],
    "filters": {},
    "seed": 7
  }
}

```

```
}
}
```

The selection must be consistent with the materialized dataset. If sampling is used, the selected IDs must exist in the generated sample.

8. Framework And Models

`framework_config` points to the framework configuration:

- DMF uses TOML files.
- Mem0 uses YAML files.

The file must provide `path`, `sha256`, `format`, and optionally `profile`.

`models.answerer` and `models.judge` define provider, model, and parameters. Example:

```
{
  "models": {
    "answerer": {
      "provider": "openai",
      "requested_model": "gpt-4.1-mini",
      "parameters": {"temperature": 0, "max_tokens": 4096},
      "runtime": {"timeout_seconds": 120, "rpm": 200, "max_retries": 5}
    },
    "judge": {
      "provider": "openai",
      "requested_model": "gpt-5-mini",
      "parameters": {
        "temperature": 0,
        "max_tokens": 4096,
        "reasoning_effort": "low"
      },
      "runtime": {"timeout_seconds": 120, "rpm": 200, "max_retries": 5}
    }
  }
}
```

Do not add `api_key`, `token`, `secret`, or similar fields to JSON files: validation rejects them.

9. Main Commands

User operations go through `dmf-benchctl`. Its current orchestrator adapter uses Docker Compose, but this detail is not part of the user contract. The controller always invokes `dmf-bench` explicitly for scientific runtime operations.

Show the controller commands:

```
dmf-benchctl --help
```

List benchmarks and frameworks:

```
dmf-benchctl runtime -- list
```

List registered datasets:

```
dmf-benchctl runtime -- list-datasets
```

Validate an already materialized configuration:

```
dmf-benchctl validate --config config/experiment.json
```


Validate after materializing the dataset from the registry:

```
dmf-benchctl validate --materialize-datasets --config config/experiment.json
```

Prepare a reproducible batch without provider calls:

```
dmf-benchctl --image ghcr.io/ORG/dmf-benchmarks:TAG prepare \
  --config path/to/experiment-a.json \
  --config path/to/experiment-b.json \
  --allow-downloads \
  --observability
```

For the four checked-in sampled smoke experiments, replace the repeated `--config` options with `--suite smoke`.

`prepare` performs the deterministic operational checks as one transaction:

1. verifies that Docker and its Compose plugin are available;
2. pulls the requested published image;
3. resolves the mutable tag to an immutable registry digest;
4. rejects an image whose platform differs from the Docker engine platform;
5. verifies that required provider credentials exist, without printing or storing them;
6. validates the resolved Compose configuration and starts the required services;
7. materializes and validates every configured dataset;
8. verifies that configurations for the same benchmark declare identical dataset and sampling policies;
9. reads the dataset registry from the selected image and rejects unpinned entries;
10. records each configuration SHA-256 and immutable dataset identity;
11. generates one batch ID and deterministic run IDs.

The command writes `.dmf-bench/prepared.json` by default. This local, Git-ignored file records the image digest, image revision, platform, configuration paths and SHA-256 digests, immutable dataset revisions and hashes, sampling declarations, requested models, preparation timestamp, batch ID, and generated run IDs. It never stores API keys. The context is written only after every preparation check succeeds.

Preparation can download datasets and models but does not invoke answer or judge providers, so it does not consume OpenAI inference tokens. `--allow-downloads` records the download policy in the context for the later run.

Manually materialize a dataset:

```
dmf-benchctl runtime -- materialize-dataset \
  --dataset-id loco-official-v1 \
  --sample-fraction 0.05 \
  --sample-unit conversation \
  --sample-seed 7 \
  --sample-rounding ceil \
  --output-dir /bench/cache/datasets/manual-loco-smoke
```

The command prints a `dataset` fragment usable in a configuration and the path to `manifest.json`.

The `runtime -- ...` escape hatch forwards advanced arguments to `dmf-bench` inside the image. Normal run, validation, state, verification, and publication operations have dedicated controller commands and should use those commands.

10. Execution

Before launching the run, set the image and provider variables required by the config, preferably in the repository `.env` file:

```
DMF_BENCH_IMAGE=ghcr.io/ORG/dmf-benchmarks:TAG
OPENAI_API_KEY=...
HF_HUB_OFFLINE=1
```

Do not set `QDRANT_URL` for the standard local stack. The benchmark container receives the internal endpoint `http://qdrant:6333` automatically.

Launch a run:

```
dmf-benchctl run --config config/experiment.json --run-id RUN_ID
```

To execute every experiment from the context produced by `prepare`, in declared order and stopping on the first failure:

```
dmf-benchctl run --prepared
```

An alternative context path can be supplied after `--prepared`. The controller uses the image digest, revision, download policy, observability choice, config paths, and generated run IDs stored by `prepare`. It checks the provider credential again immediately before execution. The secret itself remains in the host environment or repository `.env` file.

For version 2 preparation contexts, the controller verifies every config SHA-256 before starting infrastructure. The image digest fixes the built-in dataset registry, and materialization verifies the pinned source bytes.

Execution is detached by default. The controller starts a separate host worker, prints a JSON launch receipt containing its PID, every run ID, the worker log, and exact `status` and `verify` commands, then returns immediately. The worker preserves sequential execution and stops the batch on the first failure. Container output is retained in `.dmf-bench/logs/<RUN_ID>.log`; orchestration output is retained in `.dmf-bench/logs/<BATCH_ID>.worker.log`. Add `--follow` only to keep execution in the foreground and stream container logs for interactive diagnostics.

The detached prepared batch is restartable. On every invocation its worker checks each generated run ID: existing completed, incomplete, or failed runs go through the runtime resume path, while only missing runs start from an empty state. The batch still stops immediately if a resumed or new run fails.

Resume a failed run using the same image digest and preparation policies:

```
dmf-benchctl resume --prepared --run-id RUN_ID
```

Resume is quiet by default as well; add `--follow` only when live diagnostics are intentionally required.

`RUN_ID` must be unique and readable, for example:

```
official-20260809-locomo-dmf
```

The controller starts Qdrant and the read-only Artifact API, leaves them running for result inspection, and executes `dmf-bench run` in a one-shot container. Add `--observability` to start Prometheus and Grafana as well. Add `--allow-downloads` only during an intentional cache-preparation or online run.

The one-shot container receives the stable network alias expected by Prometheus. The Artifact API also exports persisted run metrics from the shared volume, so Grafana continues to show the latest progress, final token usage, timings, final process resource measurements, and evaluation values after the benchmark process exits. Qdrant must pass its healthcheck before a benchmark container can start.

During the run, JSON events are printed. A completed run exits with code 0 and prints a final `COMPLETED` state. Stop services afterward without deleting run or cache volumes:

```
dmf-benchctl stack down
```

11. Artifact API

The Artifact API exposes results in read-only mode. When started locally, it normally responds at:

```
http://127.0.0.1:8000
```

Open that address in a browser for the simplest workflow. The landing page lists committed runs and all their artifacts. Select a run, then use **View** for text-based artifacts or **Download** to save any artifact directly. The endpoints below remain useful for agents and automation.

Health check:

```
curl --fail http://127.0.0.1:8000/health
```

Run state:

```
curl --fail http://127.0.0.1:8000/runs/RUN_ID
```

Artifacts:

```
curl --fail http://127.0.0.1:8000/runs/RUN_ID/artifacts
```

The run state reports the final result and the local path to the `COMPLETED.json` marker:

```
curl --fail http://127.0.0.1:8000/runs/RUN_ID
```

To download a single artifact, use exactly one of the paths returned by `/runs/RUN_ID/artifacts`.

Cost, timing, and resource metrics:

```
curl --fail http://127.0.0.1:8000/runs/RUN_ID/artifacts/reports/usage.json
```

```
curl --fail http://127.0.0.1:8000/runs/RUN_ID/artifacts/reports/timing.json
```

```
curl --fail http://127.0.0.1:8000/runs/RUN_ID/artifacts/reports/resources.json
```

Grafana and Prometheus

When the stack was prepared or started with `--observability`, use the Operations dashboard for live and persisted runtime information:

```
http://127.0.0.1:3000/d/dmf-benchmarks/dmf-benchmarks
```

It shows input-record and evaluation-item progress, run state, LLM tokens, execution and pipeline timings, benchmark CPU and memory, and Qdrant activity. CPU is process CPU: 100% means one fully used logical CPU, so a machine using 12 logical CPUs can report about 1200% when unbounded. The standard stack caps the benchmark at `DMF_BENCH_CPUS=4.0`, so its sustained default ceiling is about 400%. The panel combines live utilization with the persisted final-attempt average; the memory panel likewise retains final peak RSS after process exit.

Final scientific metrics are deliberately separated into the Evaluation dashboard:

```
http://127.0.0.1:3000/d/dmf-benchmarks-evaluation/dmf-benchmarks-evaluation
```

The Evaluation dashboard is populated only after evaluation reports have been written; it does not expose a partial judging score during a run. Persisted values come from the shared run volume through the Artifact API and remain available after the benchmark container terminates.

At the top of both dashboards, select exactly one **Benchmark framework** (LoCoMo or LongMemEval) and one **Memory system** (DMF or Mem0). Every panel is bound to those selections; series from different benchmark and memory-system combinations are not overlaid.

Grafana starts with a dark theme. On this loopback-only stack anonymous users have the Editor role, so **Save dashboard** overwrites the dashboard copy in Grafana's persistent data volume. Those edits survive container recreation and stack shutdown. A later repository release can intentionally replace the provisioned dashboard definition. Prometheus is available at `http://127.0.0.1:9090` for advanced inspection.

12. Verification And Resume

Verify a published run:

```
dmf-benchctl verify --run-id RUN_ID
```

Inspect state:

```
dmf-benchctl status --run-id RUN_ID
```

The default output is designed for an operator and includes:

- the persisted run state;
- a plain-language description of the current lifecycle phase;
- an ASCII progress bar and percentage;
- **evaluation items**, meaning questions or records that receive a benchmark result;
- **input records**, meaning source conversations or source records already processed;
- the appropriate next action (**status** again or **verify**).

For agents and automation, request the stable JSON representation:

```
dmf-benchctl status --run-id RUN_ID --json
```

The public JSON uses `evaluation_items` and `input_records` with the explicit fields `completed`, `total`, and `failed`. It deliberately does not expose the ambiguous internal `units` terminology used by persisted checkpoints.

Resume an interrupted run:

```
dmf-benchctl resume --run-id RUN_ID
```

For an individual run, inspect its state before manually resuming it. A prepared batch performs this check automatically and safely resumes existing run IDs.

13. Official Results

The official run result is published in the `/bench/runs` volume.

The most important files are:

Artifact	Content
<code>final/COMPLETED.json</code>	Final state, counts, and main references.
<code>final/manifest.json</code>	Run publication manifest.
<code>run-manifest.json</code>	Scientific run manifest.
<code>resolved-config.json</code>	Resolved configuration without secrets.
<code>reports/usage.json</code>	Tokens and provider usage when available.
<code>reports/timing.json</code>	Timing for measured phases and operations.
<code>reports/resources.json</code>	Process CPU, average utilization, RSS, cgroup observations, and configured limits.
<code>evaluations/primary_judge_score.json</code>	Primary LLM judge aggregate.
<code>evaluations/rigorous_report.json</code>	Deterministic rigorous metrics.
<code>evaluations/ablation_report.json</code>	Retrieval ablation report, if requested.
<code>logs/events.jsonl</code>	Lifecycle JSON events.
<code>datasets/materialization-manifest.json</code>	Dataset manifest copied into the run when the dataset is materialized from the registry.

To archive an official run, publish the run OCI bundle or keep at least the image name and digest, `RUN_ID`, `resolved-config.json`, dataset materialization manifest, `final/COMPLETED.json`, `reports/usage.json`, `reports/timing.json`, and `reports/resources.json`.

Offline deterministic re-evaluation

When a deterministic evaluator changes, a committed LoCoMo run can be re-evaluated without repeating ingestion, answer generation, judging, or paid provider calls:

```
dmf-benchctl evaluate --run-id RUN_ID
```

The source run remains immutable. The command verifies it first and writes `rigorous_report.json`, `ablation_report.json`, and `derivation.json` under:

```
/bench/runs/.derived-evaluations/RUN_ID/EVALUATOR_VERSION/
```

`derivation.json` records the source final-manifest hash, source evaluation hash, evaluator version, report hashes, and a derivation fingerprint. Derived reports are listed by `/runs/RUN_ID/derived-evaluations` in the Artifact API.

14. OCI Publication

A completed run can be published to GHCR as an OCI artifact. The artifact is not an executable image: it is a complete snapshot of the run directory, meaning everything under `runs/RUN_ID/`.

Before publication, the controller asks `dmf-bench` to verify the committed run and add these files to the run directory:

File	Purpose
<code>OFFICIAL-RUN.json</code>	Bundle metadata: OCI ref, subject, image, harness commit, and run verification.
<code>SHA256SUMS</code>	SHA-256 for every run file, excluding <code>SHA256SUMS</code> itself.

Command:

```
dmf-benchctl publish-run \  
  --run-id RUN_ID \  
  --ref ghcr.io/ORG/dmf-benchmarks-runs:RUN_ID \  
  --subject ghcr.io/ORG/dmf-benchmarks@sha256:IMAGE_DIGEST
```

`--ref` is the GHCR destination for the run bundle. `--subject` links the bundle to the Docker image that produced the run.

Packaging happens inside the benchmark image against the shared run volume. The resulting archive is retained on the host, under `bundles/` by default. The controller then calls the host `oras` executable to push that exact archive. Authenticate `oras` to GHCR before publication.

To test packaging without publishing:

```
dmf-benchctl publish-run \  
  --run-id RUN_ID \  
  --ref ghcr.io/ORG/dmf-benchmarks-runs:RUN_ID \  
  --output-dir bundles \  
  --dry-run
```

A real push requires `oras` installed and authenticated to GHCR. A dry run does not require `oras` because it only verifies and exports the bundle.

15. Ready-To-Use Example

The `smoke/` directory contains ready-to-use configurations for a restricted but substantial check:

- LoCoMo with DMF: `smoke/config/experiment-locomo-dmf.json`
- LoCoMo with Mem0: `smoke/config/experiment-locomo-mem0.json`
- LongMemEval with DMF: `smoke/config/experiment-longmemeval-dmf.json`
- LongMemEval with Mem0: `smoke/config/experiment-longmemeval-mem0.json`

Run a ready config directly from the repository root:

```
dmf-benchctl run \  
  --config smoke/config/experiment-locomo-dmf.json \  
  --run-id smoke-20260809-locomo-dmf
```

The controller detects the `smoke/` path and automatically adds the required read-only mount. No orchestrator-specific option is needed.

This is only an operational example. For a real experiment, create a dedicated configuration and treat dataset, selection, models, and costs as explicit decisions.

16. Common Problems

Missing OPENAI_API_KEY. Check the host environment or repository `.env` file. Do not print the key in logs.

Config not found. For `dmf-benchctl --config`, verify the host path. For paths inside the JSON file, verify the corresponding `/bench` mount.

Dataset not found. If the config uses `registry_id`, use `run` or `validate --materialize-datasets`. If it uses `dataset.path`, check mounts and SHA-256.

SHA-256 mismatch. The file is not the one declared. Do not change the hash to make validation pass: regenerate or replace the file with the approved one.

Read-only file system. This is normal. Write only to `/bench/cache` and `/bench/runs`.

Qdrant not reachable. Run `dmf-benchctl stack status`. The standard stack injects `QDRANT_URL=http://qdrant:6333` automatically.

Download blocked. If `HF_HUB_OFFLINE=1`, Hugging Face downloads do not start. Prepare the cache before the run or change policy only for a controlled phase.

Provider rate limit or timeout. Check `logs/events.jsonl`, `reports/usage.json`, and run state before retrying.

OCI publication failed. Check that `oras` is installed, GHCR login is valid, and `dmf-benchctl verify --run-id RUN_ID` passes before pushing.

17. Operator Or Agent Checklist

1. Install `dmf-benchctl` with `pipx`; do not prepend user commands with `Poetry`.
2. Prepare `.env` without committing secrets.
3. Prepare or choose the JSON configs.
4. Run `dmf-benchctl --image IMAGE prepare --config CONFIG ...`; use `--suite smoke` for the checked-in sampled suite.
5. Review `.dmf-bench/prepared.json`, especially image digest, platform, model names, run IDs, and config list.
6. Launch the batch with `dmf-benchctl run --prepared`; paid provider calls start here.
7. Check each exit code and `COMPLETED` state.
8. Read artifacts and dataset manifests from the Artifact API.
9. Compare materialized dataset hashes between frameworks evaluating the same benchmark.
10. Compare `reports/timing.json` and `reports/resources.json` only when image, dataset materialization manifest, model settings, CPU limit, and thread limit match between DMF and Mem0.
11. Save usage, timing, resources, resolved config, and dataset manifest.
12. Publish each OCI bundle with `dmf-benchctl publish-run` if the run is official.
13. Stop services with `dmf-benchctl stack down`; named volumes are preserved.